INF3132 – Programmation Orientée Objet
S5 2023–2024

esiea

Cours 4:

Relations entre classes,
Généricité,
Collections,
Entrées/Sorties

Felicia IONASCU
Siba HAIDAR
Aylen RICCA

Plan

1. Rappels
2. Relations entre les classes
3. Généricité
4. Collections d'objets
5. Le système entrées/sorties

Plan

1. Rappels
2. Relations entre les classes
3. Généricité
4. Collections d'objets
5. Le système entrées/sorties

Rappels : encapsulation

- On regroupe les classes en package (*library units*) avec le mot-clef `package` en début du fichier
  - `package com.mycompany.myframework.complex`
- On protège l'accès aux attributs et méthodes d'une classe par des modificateurs d'accès

Modificateur d'accès	Visibilité des éléments
`Public`	Classes à l'intérieur et à l'extérieur du package
`(friendly)`	Classes à l'intérieur du package
`Protected`	Classe courante et ses classes filles, à l'intérieur et à l'extérieur du package
`Private`	Classe courante

Rappels : classe abstraite

- Le compilateur garantit qu'on ne peut pas instancier des objets d'une classe abstraite (erreur de compilation)
- Si la classe fille n'implémente pas les méthodes abstraites de la classe mère, elle est aussi abstraite

```
abstract class Forme {
     public abstract void dessine ();      //la méthode abstraite
}                                          //n'a pas d'implémentation
class Cercle extends Forme {
     public void dessine () {
               System.out.println("Cercle.dessine");
     }
}
public class Maths {
     public static void geometrie  (Forme f) {
               f.dessine();                //dessine une forme
     }
     public static void main(String[] args) {
               Forme monCercle = new Cercle();
               geometrie(monCercle);       //Upcasting
     }
}
```

Rappels : interface (1/2)

- Une interface est un autre **type abstrait**
- Elle établit un protocole de communication (un contrat)
- Elle peut être vue comme un moyen qui permet de :
  - **caractériser un comportement** ou de
  - **réaliser un groupement de services rendus**

- On déclare une interface avec le mot clef `interface` :

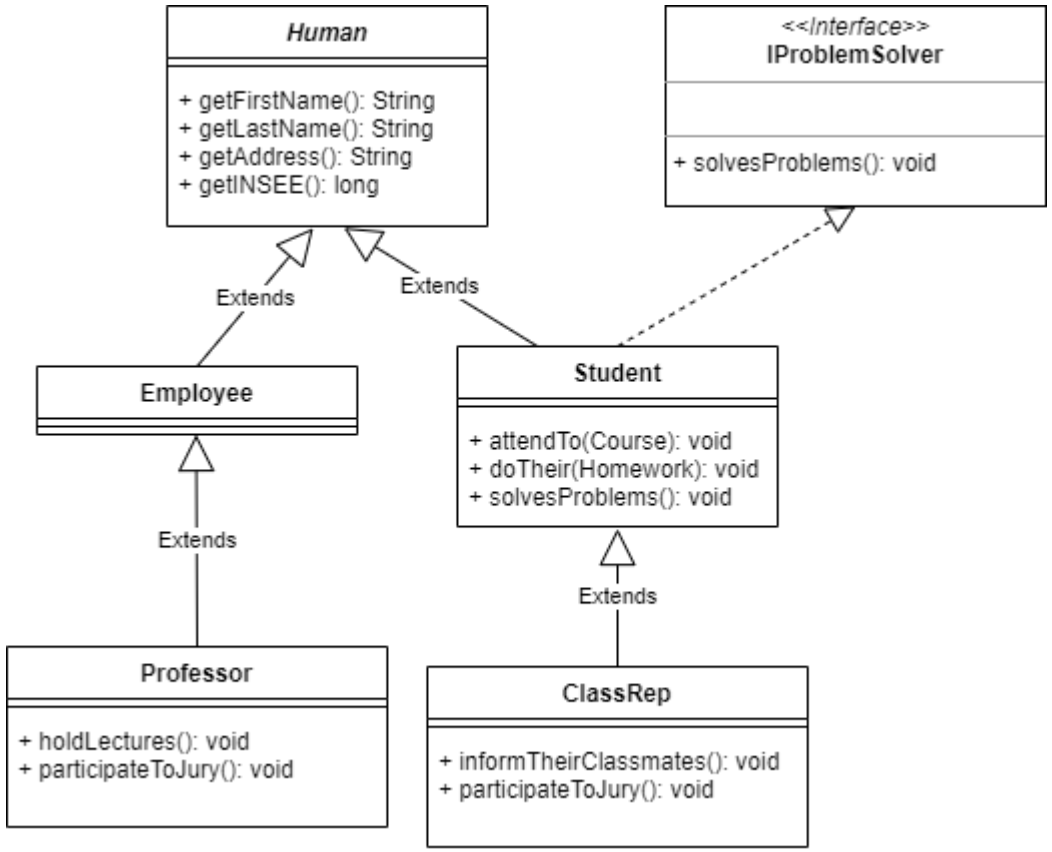
```
interface MyInterface
```

- On déclare qu'une classe implémente une interface avec le mot clef `implements` :

```
class MyClass implements MyInterface
```

- Une classe qui réalise une interface doit implémenter toutes les méthodes de l'interface

Rappels : interface (2/2)




```
abstract class Human
interface IProblemSolver
class Employee extends Human
class Student extends Human implements IProblemSolver
class Professor extends Employee
class ClassRep extends Student
```

Plan

1. Rappels
2. Relations entre les classes
3. Généricité
4. Collections d'objets
5. Le système entrées/sorties

Du simple au complexe

- L'une des caractéristiques remarquables des langages OO est la réutilisation du code = création de nouvelles classes sans changer les classes existantes

- Cela se fait par :
  - Héritage
  - **Association**
    - **Agrégation**
    - **Composition**

=> On peut créer des relations entre les classes plus ou moins fortes, de la même manière dont les objets sont interdépendants dans le monde réel

Association

- Une **relation** « faible » qu'on crée entre des objets de classes différentes

```
class School {
    private String name;
    School(String name) { this.name = name; }
    public String getSchoolName() { return this.name; }
}
class Student {
    private String name;
    Student(String name) { this.name = name; }
    public String getStudentName() { return this.name; }
}
class Association {
    public static void main (String[] args) {
      School school = new School("ESIEA");
      Student st = new Student("Nicolas");
      System.out.println(st.getStudentName() + " attends " + school.getSchoolName());
    }
}
```

Agrégation

- Relation de type « a un » (« has-a ») ; unidirectionnelle
  - Exemple : une maison a des fenêtres, un nombre romain à des chiffres romains
- Les deux classes ont du sens individuellement

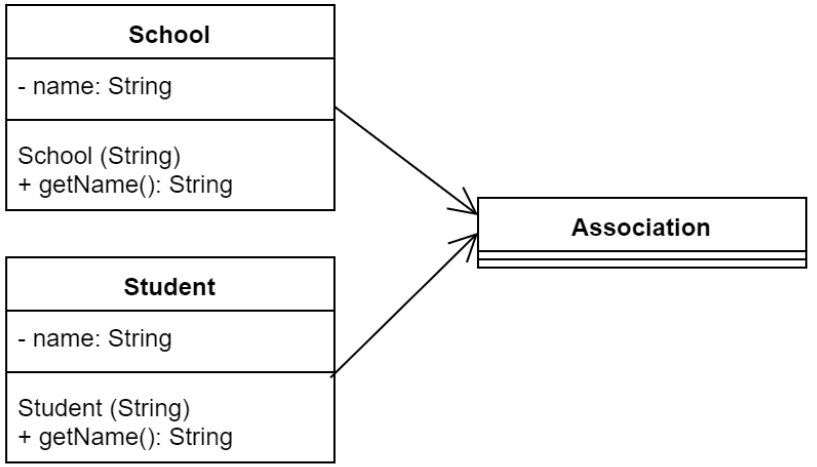
```
class Student {
   private String name;
   Student(String name) { this.name = name; }
}
class School {
   private String name;
   private List<Student> students;
   School(String name, List<Student> students) { this.name = name; this.students = students; }
}
class Aggregation {
   public static void main (String[] args)      {
      Student st1 = new Student("Alex");
      Student st2 = new Student("Julia");
      List <Student> esiea_students = new ArrayList<Student>();
      esiea_students.add(st1);
      esiea_students.add(st2);
      School school = new School("ESIEA", esiea_students);
   }
}
```

Composition

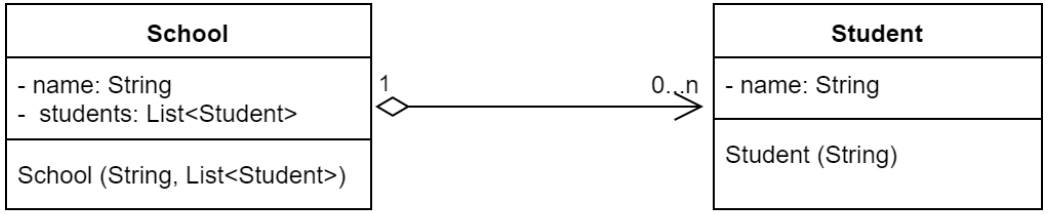
- Relation de type « fait partie de » (« part of ») ; unidirectionnelle
  - Exemple : une feuille fait partie d'un livre
- Les deux entités sont interdépendantes
  - La destruction de l'objet composé implique la destruction de ses composants

```
class BookPage {
    private String text;
    BookPage(String text) { this.text = text; }
}
class Book {
    private String name;
    private List<BookPage> pages;
    Book(String name, List<BookPage> pages) { this.name = name; this.pages = pages;}
}
class Composition {
    public static void main (String[] args)      {
        BookPage p1 = new BookPage("This is the title of the book");
        BookPage p2 = new BookPage("This is the first chapter");
        List <BookPage> pages_of_my_book = new ArrayList<BookPage>();
        pages_of_my_book.add(p1);
        pages_of_my_book.add(p2);
        Book book = new Book("My first two-page book", pages_of_my_book);
    }
}
```

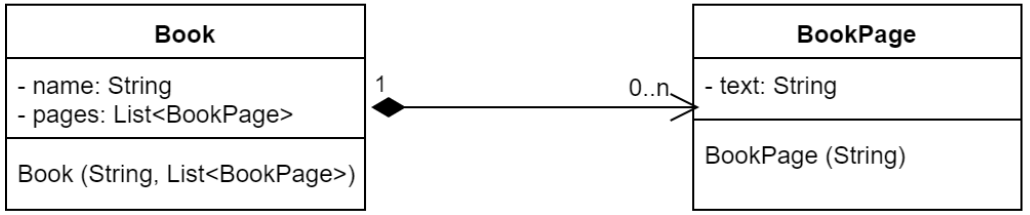
En UML



Agrégation



Composition



Exercice



Cours 4 - Question 1

Plan

1. Rappels
2. Relations entre les classes
3. Généricité
4. Collections d'objets
5. Le système entrées/sorties

Généricité

- Les ***generics*** permettent d'avoir **des types** (classes et interfaces) **en paramètre** lorsqu'on définit des classes, des interfaces et des méthodes

- Avantages :
  - Les vérifications de type se font au moment de la compilation
  - Elimination des *cast* (la vérification du type pour un *cast* se fait à l'exécution)
  - Permettent l'écriture des algorithmes génériques

  => moins de risque de bugs

Généricité

```
public class Conteneur {
    private Object object;

    public void set(Object object) {
        this.object = object;
    }
    public Object get() {
        return object;
    }
}
```

- Conventions :
  - E – type d'élément d'une collection
  - K et V – clé (key) et valeur (value) d'une paire
  - N – nombre
  - T – n'importe quel type
  - S,U,V etc. – 2nd, 3eme, 4eme types

- Avant la généricité :
  - On peut passer un objet de n'importe quel type en paramètre
  - On ne peut pas vérifier à la compilation comment on utilise cette classe

```
public class Conteneur<T> {
    //T  = "Type"
    private T t;

    public void set(T t) {
        this.t = t;
    }
    public T get() {
        return t;
    }
}
```

Généricité

```
public class Conteneur<T> {

    private T t;

    public void set(T t) {
        this.t = t;
    }
    public T get() {
        return t;
    }
}
```

- Invocation de type générique :

```
Conteneur<String> stringConteneur;
```

- Instanciation :

Avec la notation « diamant » (le compilateur déduit le type)

```
Conteneur<String> stringConteneur = new Conteneur<>();
```

Généricité

- Exemple avec deux paramètres

```
public interface Paire<K, V> {
    public K getKey();
    public V getValue();
}

public class PaireCleValeur<K, V> implements Paire<K, V> {

    private K key;
    private V value;

    public PaireCleValeur(K key, V value) {
        this.key = key;
        this.value = value;
    }

    public K getKey()   { return key; }
    public V getValue() { return value; }
}
```

```
Paire<String, Integer> p1 = new PaireCleValeur<>("PuissanceDeDeux", 1024);
Paire<String, String>   p2 = new PaireCleValeur<>("Ecole", "ESIEA");
Paire<String, Conteneur<Integer>> p = new PaireCleValeur<>("nombres", new Conteneur<>());
```

Généricité

- Méthodes génériques

```
public class PaireComparator {
    public static <K, V> boolean compare(Paire<K, V> p1, Paire<K, V> p2) {
        return p1.getKey().equals(p2.getKey()) &&
               p1.getValue().equals(p2.getValue());
    }
}
```

Inférence de type
par le compilateur

```
Paire<Integer, String> p1 = new Paire<>(1, "Volvo");
Paire<Integer, String> p2 = new Paire<>(2, "BMW");

boolean resultat = Util.<Integer, String>compare(p1, p2);

boolean resultat = Util.compare(p1, p2);
```

Généricité

- Quelques restrictions
  - On ne peut pas créer des instances de types génériques avec des types primitifs

```
Paire<int, String> p1 = new Paire<>(1, "Volvo");    => erreur
```

  - On ne peut pas créer des instances de types paramétrisés

```
public class Conteneur<T> {
    T  instance = new T();              => erreur
}
```

  - On ne peut pas declarer des attributs statiques de type parametrisé

```
public class Conteneur<T> {
    private static T attribut;          => erreur
}
```

  - On ne peut pas créer des tableaux de types paramétrisés
  - Une classe générique ne peut pas hériter de la classe `Throwable` directement ou indirectement

Exercice



Cours 3 - Question 9

Plan

1. Rappels
2. Relations entre les classes
3. Généricité
4. Collections d’objets
5. Le système entrées/sorties

Collections d'objets

- **Une collection est un concept qui modélise un regroupement de concepts**
  - Permet de regrouper, stocker, de manipuler et retrouver les objets
- Exemples de concepts qui ont du sens à être gérés ensemble :
  - Messages dans une boite à lettres électronique
  - Etudiants dans une classe
  - Sportifs inscrits au club d'athlétisme
  - Villes dans un pays
  - …
- Les collections aident à :
  - Accroître la qualité des applications
  - Réduire l'effort de programmation
  - Réutiliser le code
  - Favoriser l'inter-operabilité entre les applications

De tableau à la Collection

- Le tableau est une façon d'organiser les objets, mais :
  - Le code est peu lisible et son utilisation est complexe
  - On travaille avec les indices => source d'erreurs
  - Le code sera dupliqué pour chaque nouveau besoin

```
public class MuseumManager {
   public static void main(String[] args) {
        Museum museum = new Museum();
        museum.addPainting("Mona Lisa");
        museum.addPainting("Guernica");
        museum.addPainting("Ecole d'Athenes");
        museum.addPainting("La jeune fille a la
                                    perle");
        museum.addPainting("La ronde de nuit");
        museum.addPainting("La nuit etoilee");

        System.out.println(museum.toString());
        museum.removePainting("Guernica");
        System.out.println(museum.toString());
    }
}
```

```
public class Museum {
      private String paintings[] = new String[6];     //capacité max = 6
      private int  nPaintings = 0;
      public void addPainting(String p) {
           if (nPaintings < 6) {
                paintings[nPaintings++] = p;
            }
      }
      public void removePainting(String p) {
           for (int i = 0; i < paintings.length; i++) {
                //rechercher l'objet p dans paintings
                if (paintings[i].equals(p)) {
                      // deplacement des elements
                      for (int j = i; j < paintings.length - 1; j++) {
                            paintings[j] = paintings[j+1];
                      }
                      nPaintings--;
                      break;
                 }
            }  //combien de mémoire reste utilisée? que fait-on du dernier élément ?
      }
[...]
```

```
[...]
   public String toString() {
        String allPaintings = new String();
        for (int i = 0; i < nPaintings; i++) {
           allPaintings+=paintings[i] + "\n";
        }
        return allPaintings;
   }
}  //class Museum
```

```
D:\>java MuseumManager
Mona Lisa
Guernica
Ecole d'Athenes
La jeune fille a la perle
La ronde de nuit
La nuit etoilee

Mona Lisa
Ecole d'Athenes
La jeune fille a la perle
La ronde de nuit
La nuit etoilee
```

De tableau à la Collection

- La collection simplifie l'écriture du code traitant d'un ensemble de données
  - ⇒ le code est plus lisible et plus fiable
- L'implémentation de la collection ArrayList de la JDK est cachée

```
import java.util.*;

class MuseumCollection {
    private ArrayList<String> paintings = new ArrayList<String>();
    public void addPainting  (String  p) {
        paintings.add (p);
    }
    public void removePainting (String p) {
        paintings.remove (p);
    }
}
```

```
class MuseumManager {
    public static void main(String[] args) {
        MuseumCollection museum = new MuseumCollection();
        museum.addPainting("Mona Lisa");
        museum.addPainting("Guernica");
        museum.addPainting("Ecole d'Athenes");
        museum.addPainting("La jeune fille a la perle");
        museum.addPainting("La ronde de nuit");
        museum.addPainting("La nuit etoilee");

        System.out.println(museum.toString());
        museum.removePainting("Guernica");
        System.out.println(museum.toString());
    }
}
```

```
D:\>java MuseumManager
Mona Lisa
Guernica
Ecole d'Athenes
La jeune fille a la perle
La ronde de nuit
La nuit etoilee

Mona Lisa
Ecole d'Athenes
La jeune fille a la perle
La ronde de nuit
La nuit etoilee
```

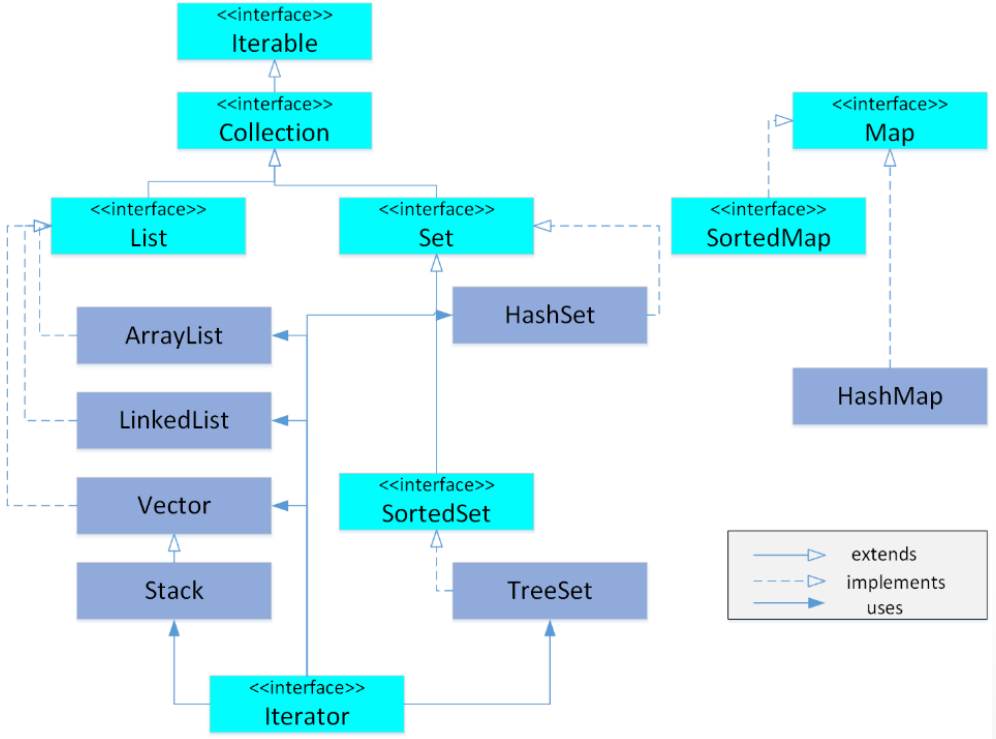
Les principales collections Java (1/3)

- Dans le package `java.util`
- Les interfaces permettent de manipuler n'importe quelle collection de manière identique

https://docs.oracle.com/javase/8/docs/api/java/lang/Iterable.html

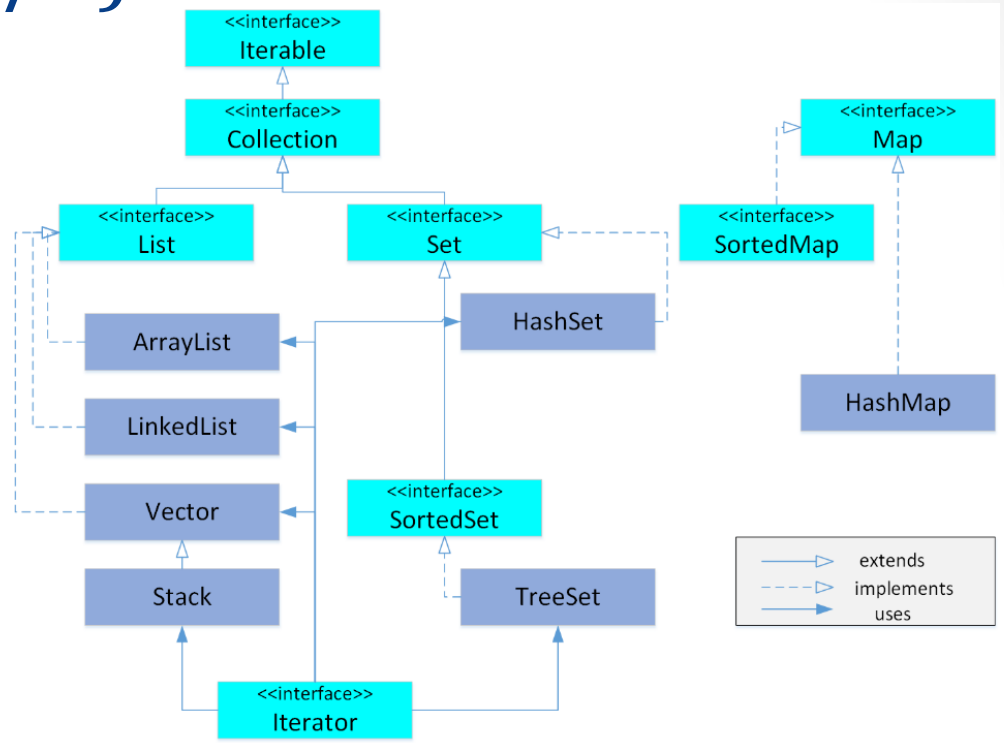
Toutes les classes qui implémentent `Iterable` peuvent être parcourues/traversées avec la boucle `forEach`

`Iterable` permet d'obtenir un `Iterator` sur les objets des collections (accès aux éléments)



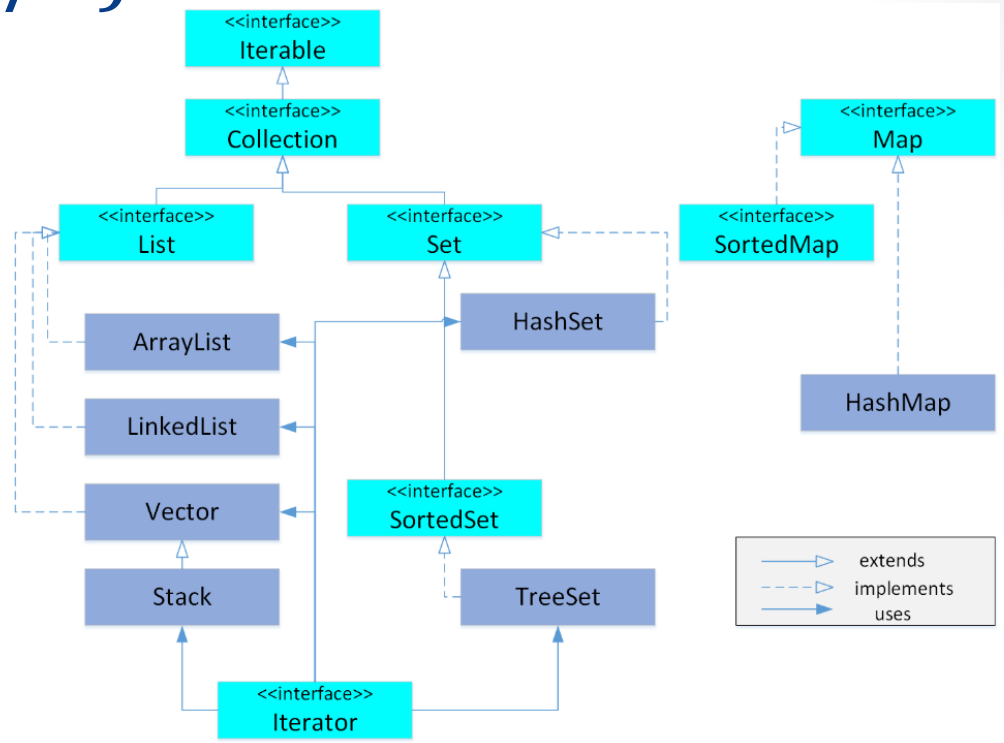
En Java, une collection est un objet contenant un ensemble de **référents sur d'autres objets**

Les principales collections Java (2/3)



- Interfaces :
  - `Set` : éléments non dupliqués et non ordonnés
  - `List` : éléments ordonnés et éventuellement dupliqués
  - `Map` : éléments indexés
  - `SortedSet, SortedMap` : maintiennent l'ordre des éléments stockés
  - `Iterator` : permet de parcourir les éléments d'une collection

Les principales collections Java (3/3)



- Classes :
  - `HashMap` : stocke des couples (clef, élément) dans une table d'index
  - `ArrayList` : stocke les éléments dans un tableau
  - `LinkedList` : stocke les éléments dans une liste chaînée
  - `HashSet` : stocke les éléments dans une `HashMap`, chaque élément étant à la fois *clef* et *index*
  - `TreeSet` : stocke les éléments dans un arbre, l'ordre est maintenu par un `SortedSet`

Les collections de type Set et List

- Set :
  - Les éléments sont uniques
  - Une implémentation du concept d'ensemble mathématique
  - **L'ordre** n'a pas de signification (=> `SortedSet` pour un set ordonné)
- List :
  - Accepte des éléments dupliqués
  - Les éléments sont **ordonnés**
  - On accède à un élément avec un indice

- `Set` et `List` adaptent automatiquement leur taille en fonction du nombre d'éléments stockés
- On utilise plus souvent `List` que `Set`

equals() et hashCode() (1/4)

- https://docs.oracle.com/javase/7/docs/api/java/lang/Object.html
- `equals()` teste si un objet est égal à un autre
  - L'implémentation par défaut dans la classe `Object` compare les références
  - Mais on a souvent besoin de comparer les états des objets (=> valeurs de leurs attributs)

```
public class Pixel {
  private int rouge, bleu, vert;

  public Pixel () {
    this.rouge = 0;     this.bleu = 0;    this.vert = 0;
  }
  public void setColor(int r, int b, int v) {
     this.rouge = r;    this.bleu = b;    this.vert = v;
  }

  @Override   //on dit ainsi qu'il s'agit de la redéfinition de la méthode equals de la classe Object
  public boolean equals (Object obj) {
      if (obj instanceof Pixel) {     //on teste d'abord s'il s'agit bien d'un objet du même type
        return ((this.rouge == ((Pixel)obj).rouge) && (this.bleu == ((Pixel)obj).bleu) &&
                (this.vert == ((Pixel)obj).vert));
      }
      else return false;
  }
}
```

cast explicit vers la classe Pixel

equals() et hashCode() (2/4)

- Un code de hachage (*hashCode*) est un entier calculé à partir d'un objet
  - => il exprime son état
- Les codes de hachage devraient être très différents entre deux objets (la probabilité qu'ils soient identiques devrait être minime)
- Le `hashCode` par défaut de tous les objets (tel que défini dans la classe `Object`) est calculé à partir de son adresse en mémoire

- On peut nous-même définir comment le `hashCode` est calculé
- Le code de la méthode `hashCode` de la classe `String` :

```
@Override
public int hashCode() {
   int hash = 0;
   for (int i = 0; i < length(); i++)
      hash = 31 * hash + charAt(i);
}
```

Si on définit `equals()` dans une classe, on doit absolument définir aussi `hashCode`

equals() et hashCode() (3/4)

```
public class Pixel {
  private int rouge, bleu, vert;
  //d'autres méthodes…

  @Override
  public boolean equals (Object obj) {
      if (obj instanceof Pixel) {   //on teste d'abord s'il s'agit bien d'un objet du même type
        return } ((this.rouge == ((Pixel)obj).rouge) && (this.bleu == ((Pixel)obj).bleu) &&
                (this.rouge == ((Pixel)obj).rouge));
      else return false;
  }
  @Override //on indique qu'il s'agit de la redéfinition de la méthode hashCode de la classe Object
  public int hashCode() {
      return 7 * rouge + 11 * bleu + 13 * vert;  //il sera différent pour des objets avec des états différents
  }
}
```

- Si les attributs de la classe sont à leur tour de type classe, alors on peut utiliser :

  `public static int Objects.hash(Object ... values)`

  - https://docs.oracle.com/javase/8/docs/api/java/util/Objects.html#hash-java.lang.Object...-

equals() et hashCode() (4/4)

- Les définitions de `equals()` et de `hashCode()` doivent être compatibles
- Si `equals` retourne `true` alors le code de hachage des deux objets doit être le même
- Les deux méthodes doivent se baser sur les mêmes attributs dans leur calcul

```
public class Main {
  public static void main(String args[]){
    Pixel p1 = new Pixel();
    Pixel p2 = new Pixel();
    p1.setColor(30, 31, 100);
    System.out.println("p1: " + p1.rouge + " " + p1.vert + " " + p1.bleu);
    p2.setColor(30, 31, 100);
    System.out.println("p2: " + p2.rouge + " " + p2.vert + " " + p2.bleu);
    System.out.println("equals: " + p1.equals(p2));
    System.out.println("hashCode: " + p1.hashCode() + " " + p2.hashCode());

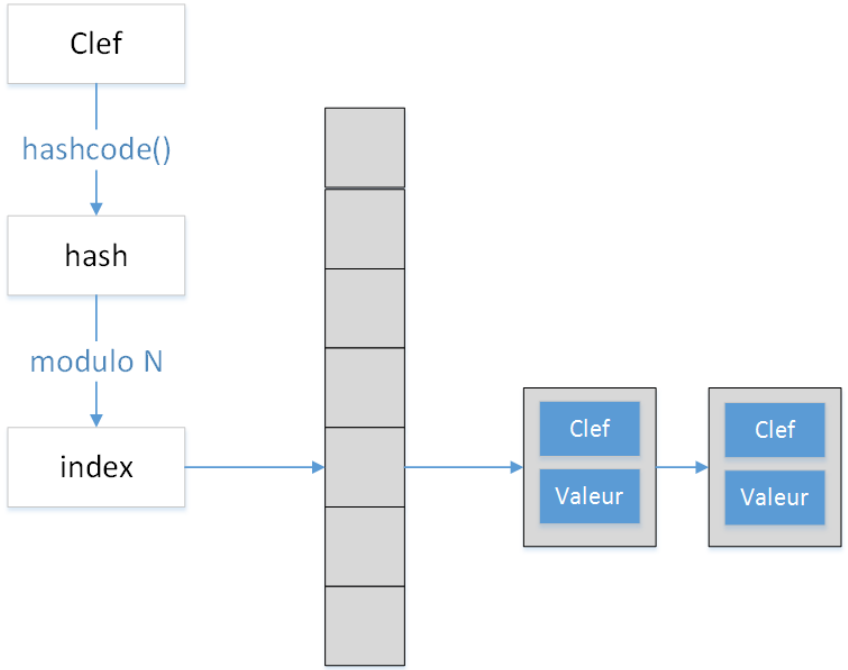
    p1.setColor(25, 2, 55);
    System.out.println("p1: " + p1.rouge + " " + p1.vert + " " + p1.bleu);
    p2.setColor(55, 2, 25);
    System.out.println("p2: " + p2.rouge + " " + p2.vert + " " + p2.bleu);
    System.out.println("equals: " + p1.equals(p2));
    System.out.println("hashCode: " + p1.hashCode() + " " + p2.hashCode());
  }
}
```

```
D:\>java Main
p1: 30 100 31
p2: 30 100 31
equals: true
hashCode: 1851 1851
p1: 25 55 2
p2: 55 25 2
equals: false
hashCode: 912 732
```

Les collections à base de table de hachage

- Permettent de stocker des paires (clef, valeur), on retrouve les valeurs en utilisant les clefs
- Très rapides

Une fonction de hachage permet de transformer une clé en une valeur de hachage, donnant ainsi la position d'un objet dans le tableau
Le calcul de la valeur de hachage :
1. La méthode `hashCode()` est utilisée pour produire un nombre entier à partir de la clé ;
2. Ce nombre entier est converti en position dans la table, en calculant le reste modulo la taille de la table.
e.g. : hashCode = 76268, 128 alvéoles
=> l'objet est placé à la position 76268 % 128 = **108**



- Deux clefs différentes peuvent produire un numéro identique => collision de hash codes (le nouvel objet sera dans ce cas-là comparé avec les objets existants)

Exemple HashMap

```
import java.util.*;

public class HashMapGenerics {
   public static void main(String args[])  {
        HashMap<String, Person> actors = new HashMap<String, Person>();
        Person henry = new Person ("Henry Fonda");
        Person jane = new Person("Jane Fonda");
        Person peter = new Person("Peter Fonda");

        actors.put("Henry", henry);
        actors.put("Jane",  jane);
        actors.put("Peter", peter);

        Set<String> keySet = actors.keySet();
        for (Iterator<String> keys = keySet.iterator(); keys.hasNext(); ) {
                String key = keys.next();
                Person person = actors.get(key);
                System.out.println(person.getName());
        }
   }
}
```

Iterable et Iterator

Toute classe qui est Iterable peut être parcourue grâce à un Iterator

- Iterator

```
void showSetContent (Set set) {
   for (Iterator it = set.iterator(); it.hasNext();) {
         Object obj = it.next();
         System.out.println(obj.toString());
   }
}
```

```
void removeSetContent (Set set) {
    Iterator it = set.iterator();
    while (it.hasNext()) {
         Object obj = it.next();
         if (obj instanceOf String) {
            if (obj.equals("object that I want to remove"))
                 it.remove();
            }
         }
     }
}
```

- Iterable (foreach)

```
List<String> cohorts =
          new ArrayList<>(Arrays.asList("1A", "2A", "3A"));

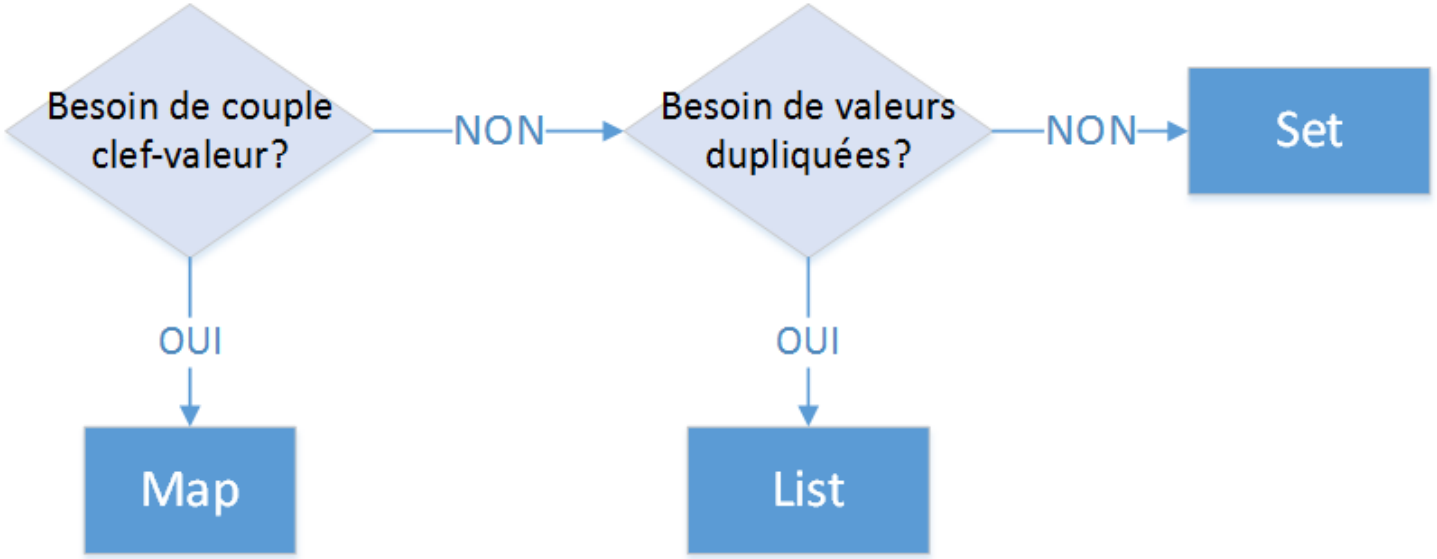
for (String cohort: cohorts) {
    System.out.println(cohort);
}
```

- Toutes les Collections implémentent les méthodes de l'interface `Iterable` : `iterator()`
- Toutes les classes qui implémentent l'interface `Iterator` implémentent les méthodes `next`, `hasNext`, `remove`, `forEachRemaining`
- Comment sait-on quelles sont ces classes dans la JDK ?

- *`Iterable`* produit une nouvelle instance d'`Iterator` sur l'appel de la méthode `iterator()`
  - `Iterator` garde l'état de l'itération (l'élément courant)
- *`Iterable`* peut parcourir la collection dans une seule direction (du premier au dernier élément)
- *`ListIterator`* peut aller du dernier au premier élément
- **On ne peut pas supprimer des éléments de la collection avec la boucle `foreach`**

Choisir une collection

- On choisit le type de collection en fonction des besoins du programme
- Les deux questions de base :



Exercice HashSet



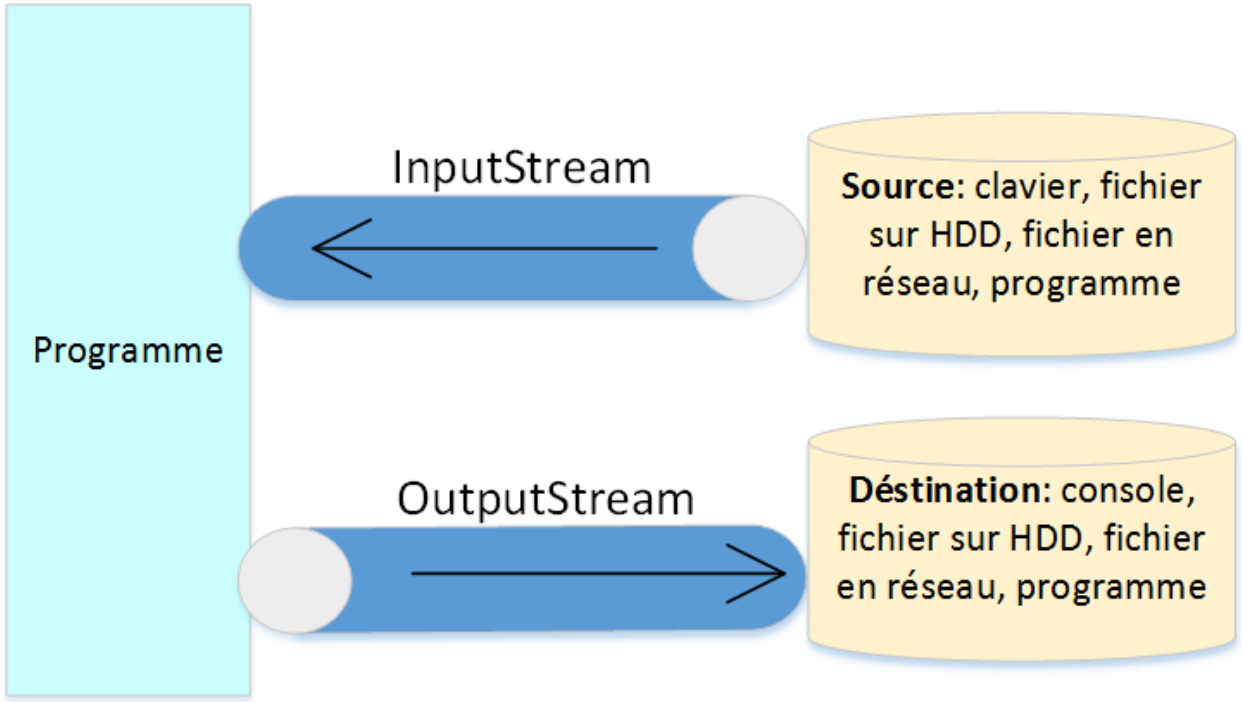
Cours 4 - Question 2 et Question 3

Plan

1. Rappels
2. Relations entre les classes
3. Généricité
4. Collections d'objets
5. Le système entrées/sorties

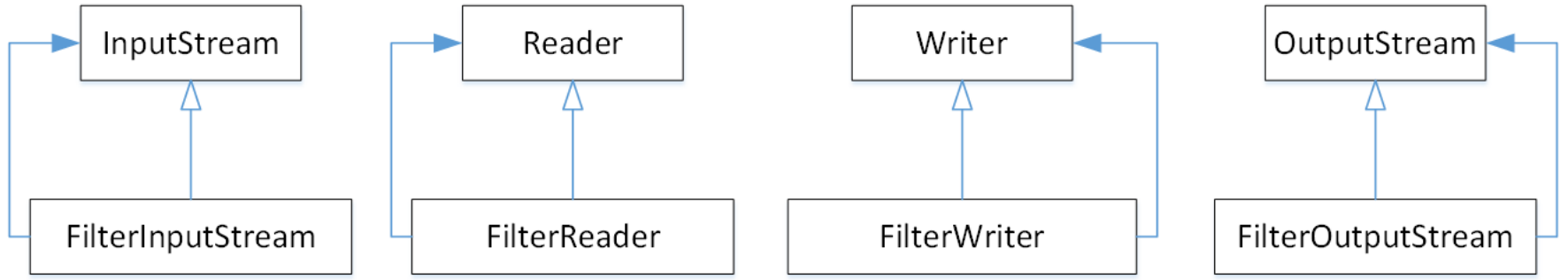
Les entrées-sorties

- Il s'agit des classes qui modélisent les interactions avec les sources/destinations de données
- Deux types de classes en Java, dans le package `java.io` :
  - Entrées/sorties par octet
  - Entrées/sorties par caractères `UNICODE`



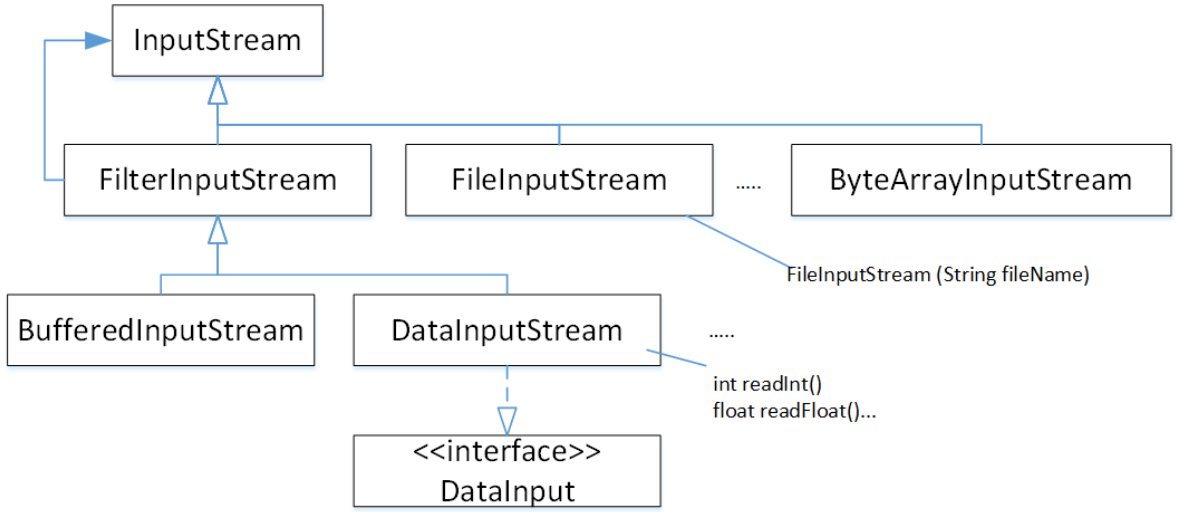
Canal et Filtre

- Un Canal (*stream*) représente une source/destination de données :
  - Un fichier, une pipe, un socket, un tableau…
  - Peut être lu et/ou écrit
  - Interface de bas niveau : lire/écrire un octet ou un caractère
- Un Filtre se connecte sur un Canal (*stream*)
  - Il permet d'offrir une **interface de haut niveau identique** à tous les canaux pour lire/écrire un entier, un flottant, une chaîne
  - Il est possible de connecter un filtre à un autre



Entrées-sorties par octet

- Les canaux :
  - `FileInputStream` : permet de lire un fichier octet par octet
  - `ByteArrayInputStream` : lit des flux d'octets provenant d'un tableau Java stocké en mémoire
- Les filtres :
  - `BufferedInputStream` : mémorise les accès à un flux d'octets (performant : les opérations se font en mémoire et non pas sur le disque)
  - `DataInputStream` : lecture des types primitifs Java (`int`, `float`…)



Exemple E/S par octet (1/3)

```
import java.io.*;

public class FileCopyWithoutBuffer {
   public static void main (String args[]) throws IOException {
       FileInputStream fis = null;
       FileOutputStream fos = null;
       try {
           fis = new FileInputStream(args[0]);   //nom du fichier en paramètre
           fos = new FileOutputStream(args[1]);  //nom du fichier en paramètre

           int b;
           while ( (b = fis.read()) != -1 )
               fos.write(b);
       } finally {
           if (fis != null) fis.close();
           if (fos != null) fos.close();
       }
   }
}
```

toujours clore les streams !

- L'exemple s'exécute très lentement. Pourquoi?

Exemple E/S par octet (2/3)

- La plupart des périphériques ne peuvent lire les données que par paquets
  - Un disque ne sait pas lire un seul octet à la fois : il ne peut lire qu'un secteur
  - Si vous lisez octet par octet, les performances sont très mauvaises car en réalité on lit à chaque requête un paquet de données

- On doit utiliser les `Buffers` (la mémoire tampon)
  - Il lit les données par paquet
  - On y accède par octet

Exemple E/S par octet (3/3)

```
import java.io.*;

public class FileCopyWithBuffer {
   public static void main (String args[]) throws Exception {
      FilterInputStream fis = null;
      FilterOutputStream fos = null;
      try {
          fis = new BufferedInputStream (new FileInputStream(args[0]));
          fos = new BufferedOutputStream (new FileOutputStream(args[1]));
          int b;
          while ( (b = fis.read()) != -1 )
                  fos.write(b);
      } finally {
        if (fis!= null) fis.close();
        if (fos!= null) fos.close();
      }
   }
}
```

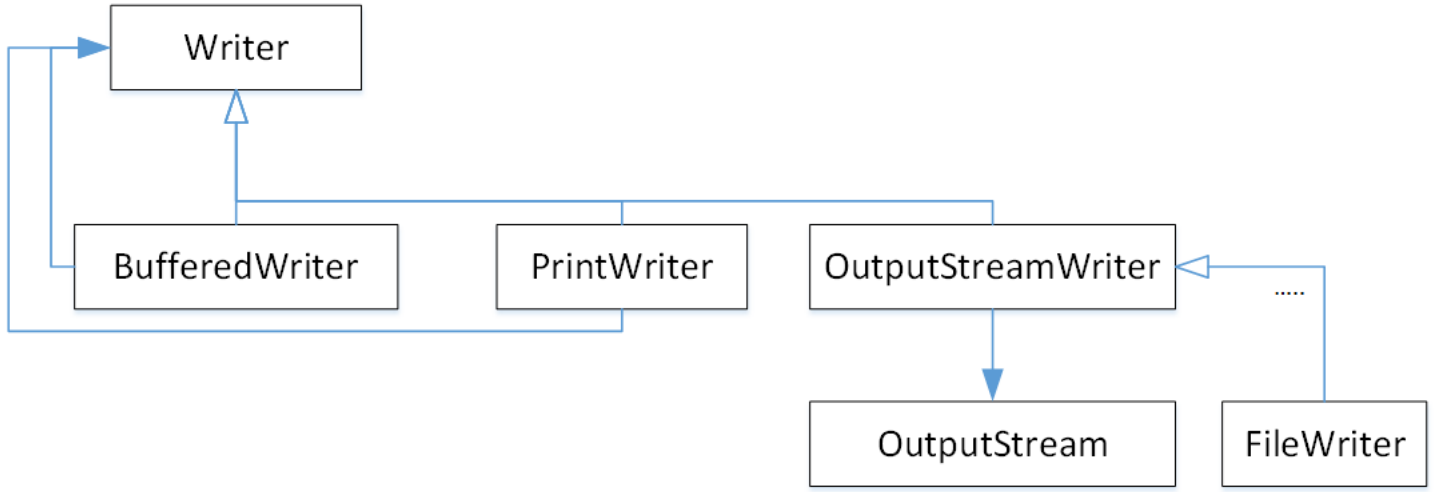
- Avec les filtres, la performance est très largement améliorée

Entrées-sorties par caractère

- Flots de caractères UNICODE-2
  - 2 octets par caractère
- Deux classes de base :
  - `java.io.Reader` : équivalent de `InputStream`
  - `java.io.Writer` : équivalent de `OutputStream`
- Les filtres existent, mais ne sont pas utilisés avec les `Readers` et les `Writers`
- Vous pouvez connecter un canal Unicode sur un canal par octets
  - `InputStreamReader` et `OutputStreamWriter`

Les Writers

- Les canaux :
  - `Writer` : classe de base permettant d'écrire un flux de caractères
  - `BufferedWriter` : mémorise le flux de caractères pour améliorer les performances. La mémoire est vidée sur le fichier lorsqu'elle est pleine
  - `OutputStreamWriter` : envoie un flux de caractères vers un `OutputStream`. Les caractères Unicode sont transformés en caractères de la plateforme (ASCII) selon une transformation qui peut être redéfinie (`CharToByteConverter`)
  - `FileWriter` : envoie un flux de caractères dans un fichier (crée automatiquement un `FileOutputStream`)



Exemple E/S par caractère

- Les méthodes `print()` et `println()` de la classe `PrintWriter` peuvent être utilisés pour écrire une chaîne, un entier
- Le « `flush` » peut être automatique ou pas

```
PrintWriter p;
try {
        FileWriter fos = new FileWriter("test.txt");
        p = new PrintWriter(fos, true);  //flush automatique
        String s = "I am a String";
        p.print(s);
        p.println(s);                    //avec retour à la ligne
} catch (IOException e) {}
finally {
        p.close();
}
```

Exercice



Cours 4 - Question 4

- BufferedReader

Feedback final

- Allez sur wooclap.com




esiea